Approximate Policy Methods in RL & PPO

Saptarishi Dhanuka, Divij Khaitan

April 12, 2025

Recap: Policy Gradients

- Policy gradients work by directly parameterising the policy using some function approximation scheme (Fourier network, MLP etc.)
- Particularly effective for continuous policy spaces
- Workhorse behind DeepSeek and ChatGPT

Policy Gradient Theorem

For any differentiable policy π, with an objective function for episode value, average reward per timestep or temporal differences J

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s, a) Q^{\pi_\theta}(s, a)]$$

Example Algorithm: REINFORCE

This uses the value of the episode v_t as an estimator for $Q^{\pi_\theta}(s, a)$

Algorithm 1 REINFORCE

function REINFORCE
  Initialise θ arbitrarily
  for each episode $\{s_1, a_1, r_2, ..., s_{T-1}, a_{T-1}, r_T\} \sim \pi_\theta$ **do**
    for $t = 1$ to $T - 1$ **do**
      $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_t, a_t) v_t$
    end for
  end for
  return θ
end function

Example Algorithm: Action-Value Actor Critic

Linear approximation for Q_w, different approximator for π_θ

Algorithm 2 QAC

function QAC
  Initialise s, θ
  Sample $a \sim \pi_\theta$
  for each step **do**
    Sample reward $r = \mathcal{R}_s^a$; sample transition $s' \sim \mathcal{P}_{s.}^a$
    Sample action $a' \sim \pi_\theta(s', a')$
    $\delta = r + \gamma Q_w(s', a') - Q_w(s, a)$
    $\theta = \theta + \alpha \nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)$
    $w \leftarrow w + \beta \delta \phi(s, a)$
    $a \leftarrow a', s \leftarrow s'$
  end for
end function

Methods: Trust Regions and Advantage Functions

- Advantage function $A(s,a) = Q(s,a) - V(s)$
- Trust region methods solve

$$\max \quad \hat{\mathbb{E}}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t] \qquad \text{(surrogate objective)} \tag{1}$$

$$\text{subject to} \quad D_{KL}(\pi_{\theta_{old}}(.|s_t)||\pi_\theta(.|s_t)) \leq \delta \qquad \text{(distance constraint)} \tag{2}$$

- Theoretical basis focuses on $\max \hat{\mathbb{E}}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t] - \beta D_{KL}(\pi_{\theta_{old}}(.|s_t)||\pi_\theta(.|s_t))$, but tuning β is impractical

Methods: Proximal Policy Optimisation

- Uses a clipped objective, where the probability ratio is clipped to a small region around 1, removing the incentive for large adjustments
- A popular method for policy gradients is to get the next t actions and back-propagate on it, so the advantage is calculated as

$$\hat{A}_t = \gamma^{T-t} V(s_T) + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t-2} r_{T-2} + \cdots + r_t - V(s_t)$$

Entropy and value estimation in PPO

- A value function $V_\theta(s_t)$ estimates the expected return from state s_t, trained using

$$L_V(\theta) = \mathbb{E}_t[(V_\theta(s_t) - V_t^{target})^2]$$

where V_t^{target} is an estimate of the true value (e.g., based on sampled rewards)

- An entropy bonus is added to encourage exploration

$$S(\pi_\theta(s_t)) = -\sum_a \pi_\theta(a|s_t) \log(\pi_\theta(a|s_t))$$

- The final objective function with hyperparameters c_1, c_2 is

$$L(\theta) = \mathbb{E}_t[L_{policy}^{CLIP}(\theta) - c_1 L_V(\theta) + c_2 S(\pi_\theta(s_t))]$$

Example Algorithm: PPO

Algorithm 3 PPO

for iteration=1, 2, ... **do**
  for actor=1, 2, ..., N **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and mini-batch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Real-world example: Robotics

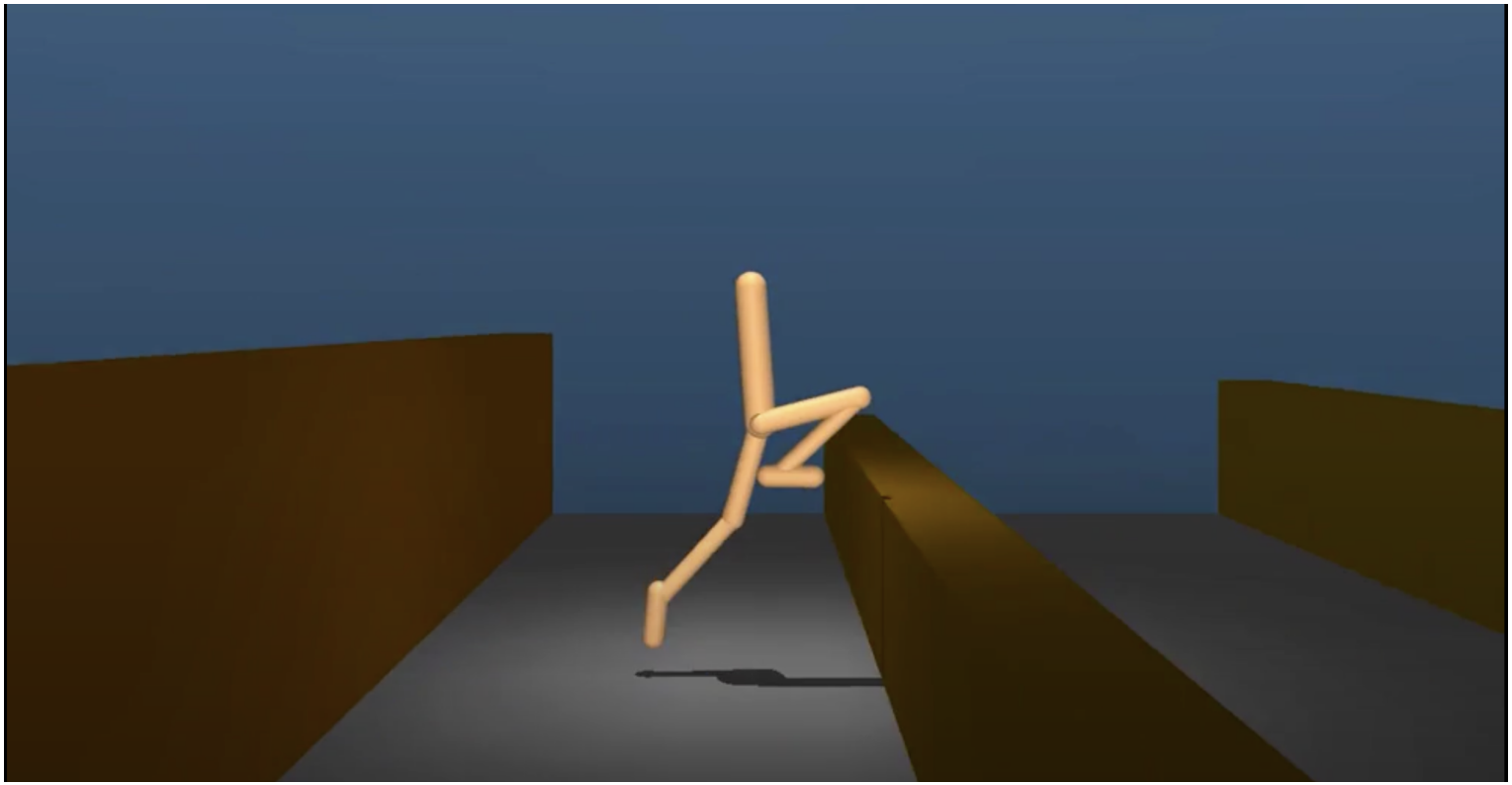

Figure: Robot Learning to Walk

Real-world example: Robotics

- We have a humanoid robot that needs to learn how to move towards a target efficiently and recover from falls
- Need speed, efficiency, robustness and adaptability, all of which are offered by PPO over other algorithms like DQN and vanilla policy gradient methods
- Here the **states** would be a combination of the angles, velocities, position, obstacle locations, etc

Real-world example: Robotics

- During training, the robot interacts with the environment and sensors capture high-dimensional state information, which it needs to map to continuous actions
- Rewards given based on how well objective being satisfied (of moving towards target, maintaining balance, etc)
- Through PPO, the policy i.e. what actions are chosen, is updated over each batch of experiences, so it takes into account immediate feedback and cumulative experience
- Clipping ensures that the changes in the robots actions aren't too drastic, which is desirable since a sudden change in actions can lead to instability

Real-world example: Robotics

- The advantage function $\hat{A}_t$ here tells us what action such as steering to the right or left is better, compared to a given baseline, on the basis of which slight adjustments can be made
- While the learning process is slow due to clipping, eventually with proper tuning the robot can learn a good policy

Real-world example: Robotics

Watch Video Here!

Real-world example: ChatGPT!

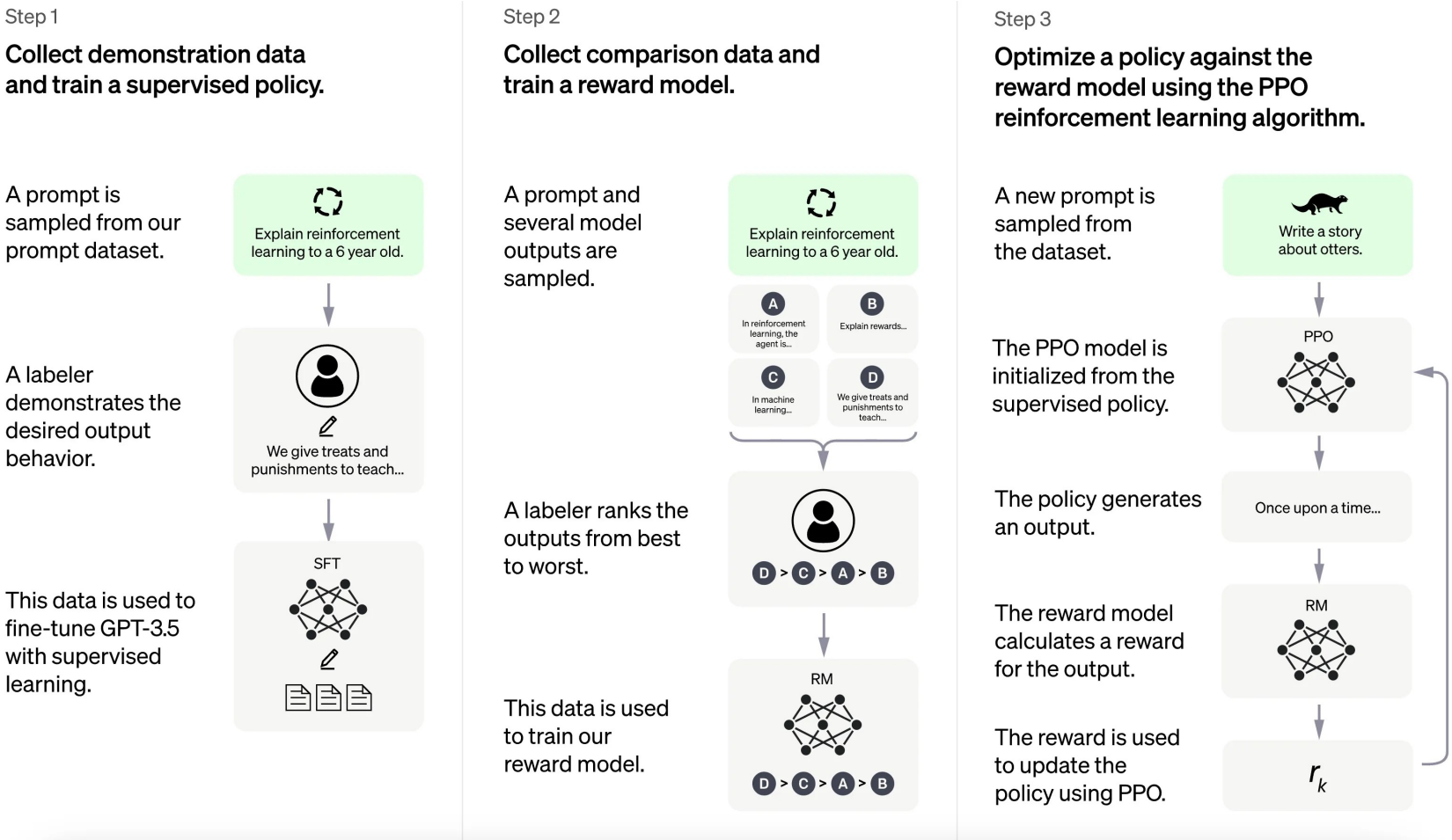

Figure: RLHF Process

Real-world example: ChatGPT!

- Pre-training and supervised finetuning are not enough to make the useful LLMs that we encounter today.
- We need **alignment** with what users want
- Human evaluators give feedback to responses, which acts as a reward signal
- This then influences a change in policy i.e. a probability distribution over the text response so that it becomes more tuned to what the evaluators prefer

Real-world example: ChatGPT!

- The change in policy is dictated by PPO by optimizing the surrogate objective with clipping, which ensures conservative policy updates
- This stops the model from over-reacting to human feedback so that it doesn't "forget" what it has learnt till that point
- This also preserves the core functionality of an LLM i.e. language generation, while still improving w.r.t user preferences and mitigation of unwanted responses